Instructional designer identifies the problem or need, clarifies the target audience, establishes instructional objectives, creates a list of tasks/knowledge/skills that will be taught.

Problem/Need:

Teaching depth first search algorithm with graph data structure, to people with little to no knowledge of the data structure or algorithm.

Target audience:

2nd year computer science students who are starting to learn data structures and algorithms, meaning users have no prior knowledge of data structures and algorithms.

Instructional objectives:

Based on the four skill levels (Informed by 'Revised Bloom's Taxonomy') which were adopted for use in CS2023.

1. Foundational Graph Concepts

Objective: Identify and describe the fundamental components of a graph structure (vertices/nodes, edges, directed vs undirected graphs) so that graph terminology is correctly interpreted without needing prior data structure experience.

ADDIE Analysis Rationale: Addresses the learners' lack of prior knowledge by establishing baseline conceptual schema before introducing algorithmic steps.

Structure Alignment:

a. *Action Verb:* Identify / Describe (Categorised under the Explain skill level for Remember & Understand cognitive processes)
b. *Topic:* Basic graph structures and components
c. *Mastery:* Correctly labeling and distinguishing nodes, edges, and edge directions in example diagrams.

2. Graph Representation

- **Objective: Translate and illustrate** how visual graph connections can be represented in simple tabular or list forms (such as adjacency lists or matrices) to track node connections.
- **ADDIE Analysis Rationale:** Prepares non-programmers to visualise how a computer or algorithm keeps track of neighboring nodes without diving into complex implementation details.
- **Structure Alignment:**
    a. *Action Verb:* Translate / Illustrate (Categorised under the Explain skill level in computing curricula)
    b. *Topic:* Graph representations (adjacency list vs matrix)
    c. *Mastery:* Accurately mapping a visual graph into a basic adjacency list representation.

3. Depth-First Traversal Mechanics

- **Objective: Trace and execute** step-by-step the traversal path of a Depth-First Search (DFS) algorithm on a given graph, exploring as far as possible along each branch before backtracking
- **ADDIE Analysis Rationale:** Builds procedural understanding of the core DFS behavior (diving deep along a single path first).
- **Structure Alignment:**
    a. *Action Verb:* Trace / Execute (Verbs under the Apply skill level in computing taxonomy)
    b. *Topic:* Depth-First Search traversal path
    c. *Mastery:* Step-by-step listing of the exact order of visited nodes on a given graph without skipping connected edges

4. Traversal State & Backtracking Logic

- **Objective: Track and record** visited versus unvisited nodes during a DFS traversal, demonstrating how backtracking allows the search to resume from the last unexplored node.
- **ADDIE Analysis Rationale:** Reinforces how algorithms avoid infinite loops in cyclic graphs and recover when hitting a dead end.
- **Structure Alignment:**
    a. *Action Verb:* Track / Record (Execution and analysis verbs aligned with Apply / Analyse cognitive dimensions)
    b. *Topic:* Visited node tracking and backtracking mechanics
    c. *Mastery:* Correctly identifying dead-end nodes and marking visited states during manual traversal simulations.

5. Search Behavior Comparison

- **Objective: Compare and contrast** the exploration pattern of Depth-First Search against Breadth-First Search (BFS) on simple graph examples to explain why DFS moves deep into a graph rather than expanding level-by-level.
- **ADDIE Analysis Rationale:** Evaluates conceptual understanding by placing DFS in context with alternative search strategies.
- **Structure Alignment:**
    a. *Action Verb:* Compare / Contrast (Verbs under the Evaluate skill level in computing curricula)
    b. *Topic:* DFS vs BFS traversal characteristics
    c. *Mastery:* Articulating the key difference between deep branch-by-branch search and level-by-level search.

Knowledge to be taught:

Graph - Depth first search algorithm

What assessment methods or tools could be used to measure the learners' achievement of the desired outcomes?

- *"app should include some form of assessment that can be used to provide the student with additional scaffolding if they are not comfortable with the work" - spec*

According to assessment guidelines, learning outcomes are mapped against Bloom's Taxonomy / CS2023 skill levels (Explain, Apply, Evaluate) and evaluated via formative (low-stakes monitoring) and summative (high-stakes evaluation) strategies

- Formative Assessment Tools (Monitoring & Feedback During Learning):
    - **Computer-Assisted Instruction (CAI) & Interactive Simulators:** Software classified under simulation and drill-and-practice taxonomies provides interactive visual environments. These apps allow learners to execute algorithmic steps, trace graph pathways, and manipulate visual models while the system automatically tracks errors and performance
    - **Adaptive Learning Systems (potentially AI-based):** Intelligent tutoring environments provide dynamic personalisation by evaluating student responses in real-time and adjusting problem difficulty or scaffolded hints based on learner performance.
    - **Interactive Concept Mapping (CAI/Simulation example):** Students are given visual graph diagrams and asked to label components (nodes, edges, directed/undirected arrows) or translate a visual graph into a basic adjacency list.
    - **Step-by-Step Algorithm Tracing Logs (CAI/Simulation example):** Digital tracking sheets where students manually record node states (`Unvisited`, `Visited`, `Current`) and stack contents as they trace a Depth-First Search execution step-by-step.
- Summative Assessment Tools (Measuring Mastery):
    - **Criterion-Referenced Written & Practical Test:** A structured assessment evaluating student performance against predefined rubric criteria (using pass descriptors and differentiated descriptors).
    - **Algorithm Comparison & Scenario Tasks:** Short-answer evaluation items where students are presented with simple search scenarios and asked to compare DFS exploration patterns against Breadth-First Search (BFS).

How will the assessment align with the instructional objectives?

Objective 1: Foundational Graph Concepts (Explain Level – Remember & Understand)

- **Instructional Objective:** Identify and describe the fundamental components of a graph structure (vertices/nodes, edges, directed vs. undirected).
- **Software Assessment Tool:** Quiz Module
- **Assessment Format:** Formative interactive labeling or drag-and-drop quiz items.
- **Alignment Rationale:** Aligns with the *Explain* skill level (*identify, describe*). The LMS automatically grades diagram-labeling items to provide low-stakes feedback, ensuring baseline terminology is mastered before moving to algorithms.

Objective 2: Graph Representation (Explain Level – Understand)

- **Instructional Objective:** Translate and illustrate how visual graph connections can be represented in simple tabular or list forms (adjacency lists or matrices).
- **Software Assessment Tool:** Interactive Math & STEM Assessment Tool/module.

- **Assessment Format:** Formative online matrix/table entry exercises.
- **Alignment Rationale:** Aligns with the *Explain* skill level (*translate, tabulate*).(E.g **Numbas** allows students to input structured adjacency lists based on visual graph prompts, automatically checking for structural accuracy.)

Objective 3: Depth-First Traversal Mechanics (Apply Level – Execution)

- **Instructional Objective:** Trace and execute step-by-step the traversal path of a DFS algorithm on a given graph.
- **Software Assessment Tool:** Interactive Graph Simulation Software / CAI Tool.
- **Assessment Format:** Formative/Summative interactive algorithm tracing simulation.
- **Alignment Rationale:** Aligns with the *Apply* skill level (*trace, execute*). The interactive simulator requires the student to click nodes in the order DFS would visit them; the app records the step-by-step sequence and checks it against the correct path execution.

Objective 4: Traversal State & Backtracking Logic (Apply / Analyse Level)

- **Instructional Objective:** Track and record visited versus unvisited nodes during a DFS traversal, demonstrating how backtracking operates.
- **Software Assessment Tool:** AI-Based Adaptive System / Intelligent Tutoring Module.
- **Assessment Format:** Formative interactive state-tracking and stack-simulation exercises.
- **Alignment Rationale:** Aligns with the *Apply* skill level (*track, record*). The adaptive tool prompts learners to update node state flags (`Visited` / `Unvisited`) and pop stack frames when hitting dead ends, offering dynamic hints if the user gets stuck in an infinite loop.

Objective 5: Search Behavior Comparison (Evaluate Level – Assessment)

- **Instructional Objective:** Compare and contrast the exploration pattern of DFS against Breadth-First Search (BFS) on simple graph examples.
- **Software Assessment Tool:** *Perhaps Numbas (https://numbas-lti.up.ac.za/)*
- **Assessment Format:** Summative short-answer prompt.
- **Alignment Rationale:** Aligns with the *Evaluate* skill level (*compare, contrast*). Learners submit a qualitative synthesis explaining why DFS explores deep branches while BFS explores level-by-level, evaluated against criterion-referenced pass descriptors.

Reference:
*Amruth N. Kumar, Rajendra K. Raj, Sherif G. Aly, Monica D. Anderson, Brett A. Becker, Richard L. Blumenthal, Eric Eaton, Susan L. Epstein, Michael Goldweber, Pankaj Jalote, Douglas Lea, Michael Oudshoorn, Marcelo Pias, Susan Reiser, Christian Servin, Rahul Simha, Titus Winters, and Qiao Xiang. 2023\. *Computer Science Curricula 2023*. ACM Press, IEEE Computer Society Press and AAAI Press. DOI: https://doi.org/10.1145/3664191*

Designer plans instructional activities to meet the objectives and teach the content.

Instructional Strategy Design

To effectively teach Depth-First Search (DFS) to beginners with no algorithm background, the instructional sequence integrates Gagné's Nine Events of Instruction and Merrill's First Principles of Instruction:

- Problem-Centered Activation (Merrill's Principle / Gagné Event 3): Begin by activating learners' prior intuitive knowledge (e.g., navigating a maze or puzzle) before formalising concepts into graphs and algorithms.
- Demonstration of Component Skills (Merrill's Principle / Gagné Event 4): Present visual demonstrations showing how nodes, edges, and visited flags function step-by-step before requiring students to execute traversal independently.
- Guided Application & Immediate Feedback (Gagné Events 5 & 7 / Flow Theory): Provide scaffolded interactive exercises where learners trace DFS. Providing frequent, clear, and immediate performance feedback maintains Csíkszentmihályi's Flow state (balancing challenge with learner skill so students remain engaged without experiencing anxiety or frustration).
- Incorporating Intrinsic Motivation (Malone & Lepper Framework): Challenge: Set clear, visible short-term goals (e.g., "Find a path from Node A to Node F without getting stuck in a loop") with optimal difficulty levels.
    - Curiosity & Fantasy: Frame the graph traversal as an interactive maze-solving scenario (fantasy element) that highlights incomplete knowledge cues.
    - Control:Give learners full interactive control to select neighboring nodes and observe the consequences on the call stack and visited markers.

Interaction & UX Design Constraints

From a **User Experience (UX) and Cognitive Ergonomics** perspective:

- **Working Memory Limitations:** Human working memory has a strictly limited capacity and duration unless actively rehearsed. To prevent cognitive overload in non-programmers, the interface must visually externalise the algorithm's state (displaying the stack and visited list on-screen) so learners do not have to hold multiple memory items in their heads simultaneously.

Frame-by-Frame storyboard layout description

Frame 1: Graph Concepts & Terminology (Explain Level)

- **Visual Layout:** Interactive canvas showing a simple, unweighted graph diagram alongside an inspector panel.
- **Instructional Action:** Students click on vertices and edges to reveal their definitions (e.g., "Node A", "Directed Edge A → B").
- **Gagné Event / Objective Alignment:** *Gain Attention & Inform Learners of Objectives*. Maps to **Objective 1** (Foundational Graph Concepts).

Frame 2: Visual Graph to Adjacency List (Explain Level)

- **Visual Layout:** Split screen with the graph on the left and an interactive drag-and-drop table on the right.
- **Instructional Action:** Students drag node connections into the adjacency list grid to build a structural representation.
- **Gagné Event / Objective Alignment:** *Present Stimulus Material & Stimulate Recall*. Maps to **Objective 2** (Graph Representation).

Frame 3: Interactive DFS Traversal & Call-Stack Simulation (Apply Level)

- **Visual Layout:** Graph on the left, an animated **Call-Stack container** on the bottom right, and a **Visited Nodes panel** on the top right.
- **Instructional Action:** The student is prompted: *"Start at Node A. Click the next node DFS will visit."* As nodes are clicked, the system pushes nodes onto the stack display, colors visited nodes green, and logs the path.
- **Gagné Event / Objective Alignment:** *Provide Learner Guidance & Elicit Performance*. Maps to **Objective 3** (DFS Traversal Mechanics).

Frame 4: Dead-End Handling, Backtracking & Comparison (Apply / Evaluate Level)

- **Visual Layout:** Traversal simulator showing a graph path that hits a dead-end (a node with no unvisited neighbors).
- **Instructional Action:** The tool highlights the dead end and prompts: *"No unvisited neighbors found. What action must DFS take?"* The learner interacts with the stack to pop the current node and backtrack to the previous decision point. Finally, a side-by-side replay contrasts the deep branch path of DFS with the level-by-level expansion of BFS.
- **Gagné Event / Objective Alignment:** *Provide Feedback, Assess Performance & Enhance Retention/Transfer*. Maps to **Objectives 4 & 5** (Backtracking Logic & Search Behavior Comparison).

References:
https://educationlibrary.org/merrills-principles-of-instruction/
https://elearningindustry.com/merrills-principles-instruction-definitive-guide

ADDIE Model – Phase 2: Design

2.1 INTRODUCTION

The Design phase of the ADDIE model focuses on planning how the educational application will help students learn graph data structures and the Depth-First Search (DFS) algorithm. This phase builds on the Analysis phase, which identified the target audience, learning needs and instructional objectives.

The application is intended for second-year Computer Science students enrolled in COS212 who have little or no prior knowledge of graphs and DFS. Therefore, the application will be designed to introduce concepts in a simple and structured way before students attend their lectures.

The proposed design will use short explanations, visual diagrams, step-by-step demonstrations, interactive activities and assessments to support learning. Students will be given opportunities to practise what they have learned and receive additional support when they experience difficulties.

2.2. Instructional Strategy

The application will follow a step-by-step learning approach that introduces basic concepts before moving to more complex topics.

Students will begin by learning the fundamental components of a graph, followed by graph representations, DFS traversal and backtracking. A short comparison between DFS and Breadth-First Search (BFS) will also be included to help students understand how DFS differs from another graph traversal algorithm.

The learning process will follow five main steps: explanation, demonstration, practice, assessment and feedback.

Each learning section will include a short explanation of the topic, a visual example and an activity that allows students to apply what they have learned. Students will then complete an assessment to check their understanding.

Students who struggle will receive hints and explanations and will be allowed to repeat activities before moving to more difficult tasks.

2.3. Learning Content and Activities

The application will contain five learning sections based on the instructional objectives identified during the Analysis phase.

2.3.1. Introduction to Graphs

The first section will introduce students to the basic components of a graph, including vertices (nodes), edges, directed graphs and undirected graphs.

Simple diagrams will be used to explain how nodes are connected and how directed and undirected graphs differ.

Students will complete an activity where they identify nodes and edges in a given graph. A short quiz will be used to assess their understanding of these concepts.

If a student answers incorrectly, the application will provide an explanation and highlight the relevant graph components.

2.3.2 Graph Representation

The second section will explain how graphs can be represented using adjacency lists and adjacency matrices.

Students will be shown examples of graphs and how their connections can be represented using lists and tables.

An interactive activity will allow students to identify neighbouring nodes and complete missing information in an adjacency list.

The application will provide immediate feedback and explain incorrect answers to help students understand how graph representations work.

2.3.3 Depth-First Search (DFS)

The third section will introduce the Depth-First Search algorithm and explain how it is used to traverse a graph.

Students will learn that DFS explores one path as far as possible before returning to explore other paths.

The application will provide a step-by-step visual demonstration of DFS using a simple graph. After observing the demonstration, students will complete an interactive activity where they select nodes in the correct traversal order.

Different colours and labels will be used to distinguish between visited nodes, unvisited nodes and the current node.

The application will use a consistent rule for choosing which neighbouring node to visit first, such as alphabetical order, so that students can follow the expected traversal sequence.

If a student selects an incorrect node, the application will provide a hint or explanation to guide them.

2.3.4 Visited Nodes and Backtracking

The fourth section will focus on how DFS keeps track of visited nodes and uses backtracking when a path cannot be explored further.

Students will observe examples showing how DFS returns to previous nodes when there are no unexplored neighbouring nodes.

Interactive activities will allow students to identify visited and unvisited nodes and determine when backtracking should occur.

If students make mistakes, the application will explain why backtracking is necessary and demonstrate the correct step.

2.3.5 Comparing DFS and BFS

The fifth section will introduce a basic comparison between Depth-First Search and Breadth-First Search.

Students will observe visual examples showing how the two algorithms traverse the same graph.

The application will explain that DFS explores one path before backtracking, while BFS explores nodes level by level.

Students will complete a short assessment where they identify the differences between the two traversal methods.

The main focus of the application will remain on DFS, while BFS will be included only to support comparison and understanding.

2.4. **Learning Theory: Constructivism**

The application will use Constructivism as its main learning theory.

Constructivism explains that students develop their understanding by actively participating in learning activities and connecting new information to what they already know.

The application will apply this theory by allowing students to interact with graph diagrams and practise the DFS algorithm instead of only reading explanations.

For example, students will be able to select nodes in a graph, follow traversal paths and observe how the algorithm moves through the graph. They will also complete activities that require them to identify graph components and determine when backtracking should occur.

These activities will allow students to explore the concepts, make decisions and learn from their mistakes.

The application will also provide guided examples before students attempt activities independently. This will help students develop their understanding gradually.

Constructivism was selected because the application aims to encourage active participation and practical learning rather than passive reading.

2.5. Assessment design

Assessment will be integrated throughout the application to monitor learning progress and identify areas where students require additional support.

Two assessment approaches will be used.

Formative assessment will occur during learning activities through short quizzes, diagram-labelling tasks and guided DFS simulations. These assessments will provide immediate feedback and opportunities for correction.

Summative assessment will take the form of a final quiz or practical traversal exercise covering the instructional objectives.

The final assessment will include questions on graph terminology, graph representation, DFS traversal, visited-node tracking and the distinction between DFS and BFS.

Assessment results will be used to provide feedback and recommend topics for revision.

2.6. Feedback and Scarfolding

The application will use scaffolding as a teaching strategy to support students who experience difficulties during learning.

Scaffolding involves providing additional guidance to help students complete tasks they may not yet be able to complete independently.

The application will use a progressive feedback approach.

When a student provides an incorrect answer for the first time, the application will display a short hint encouraging them to reconsider their answer.

If the student makes another incorrect attempt, a more detailed explanation or visual clue will be provided.

After a third incorrect attempt, the application will demonstrate the correct solution step by step and allow the student to practise again.

For example, if a student selects an incorrect node during DFS traversal, the application may remind them that DFS explores one path before moving to another. If the student continues to struggle, the application may highlight the current node and its unexplored neighbours.

This approach will help students understand their mistakes while encouraging them to complete activities independently.

If the student continues to struggle, the application will highlight the current node and the available unexplored neighbours and preventing students from becoming stuck.

This approach is intended to support independent problem-solving while preventing students from becoming stuck.

2.7. User Interface and Navigation Design

The application will use a simple and consistent interface suitable for students encountering the material for the first time.

The proposed interface will include:

- A navigation panel displaying the available learning modules.
- A central learning area containing explanations and graph visualisations.
- Interactive controls for progressing through DFS simulations.
- A progress indicator showing completed learning activities.
- Assessment sections with immediate feedback.
- A hint button providing additional assistance.

The interface will use clear labels, readable text and consistent visual indicators to distinguish visited, unvisited and currently selected graph nodes.

Students will be able to revisit earlier modules when additional revision is required.

2.8 Learner Motivation and Engagement

The application will encourage learner engagement through interactive activities, visible progress and constructive feedback.

Students will be able to observe their progress as they complete learning modules and assessments.

Short activities will be used to maintain attention, while gradually increasing task difficulty will provide opportunities for students to develop confidence.

Positive feedback will acknowledge successful completion of tasks without discouraging students who make mistakes.

The overall design aims to create a supportive environment where students can practise and improve before attending their COS212 lecture.

2.9 Expected Design Outcomes

At the end of the learning experience, students should be able to identify graph components, interpret simple graph representations, execute DFS traversal, explain backtracking and distinguish DFS from BFS.

The proposed design supports these outcomes through structured lessons, visual demonstrations, interactive activities and assessments aligned with the instructional objectives.

Designer creates and/or gathers the materials that will be used in the instruction.

Instruction is implemented in the setting it was designed for.

Instruction is evaluated according to its usefulness and effectiveness, formative (before implementation) & summative (evaluates the unit of instruction).